

Hyperledger Fabric provides three ordering mechanisms — SOLO, Kafka and Simplified Byzantine Fault Tolerance (SBFT), though the last mechanism has not yet been implemented in Fabric v1.0.

SOLO is the ordering mechanism most typically used by developers experimenting with Hyperledger Fabric networks. It involves a single ordering node.

Kafka is the Hyperledger Fabric ordering mechanism that is recommended for production use. It uses Apache Kafka, an open source stream-processing platform that provides a unified, high-throughput, low-latency platform for handling real-time data feeds. In this case, the data consists of endorsed transactions and RW sets. The Kafka mechanism provides a crash fault-tolerant solution to ordering.

Simplified Byzantine Fault Tolerance or SBFT is an ordering mechanism that is both crash fault-tolerant and Byzantine fault-tolerant. This means that it can reach agreement even in the presence of malicious or faulty nodes. The Hyperledger Fabric community has not yet implemented this mechanism, but it is on its roadmap.

These three ordering mechanisms provide alternate methodologies for agreeing on the order of transactions.

Identity verification

In addition to the multitude of endorsement, validity and versioning checks that take place, there are also ongoing identity verifications happening during each step of the transaction flow. Access control lists are implemented on the hierarchical layers of the network (from the ordering service down to channels) and payloads are repeatedly signed, verified and authenticated as a transaction proposal passes through the different architectural components.

Channels

Channels allow organisations to use the same network, while maintaining separation between multiple blockchains. Only the members of the channel on which the transaction was performed can see the specifics of the transaction. In other words, channels partition the network in order to allow transaction visibility for stakeholders only. This mechanism works by delegating transactions to different ledgers. Only the members of the channel are involved in consensus, while other members of the network do not see the transactions on the channel.

Figure 1 shows three distinct channels — blue, orange, and grey. Each channel has its own application, ledger and peers. The peers could belong to multiple networks or channels. Those that do participate in multiple channels simulate and commit transactions to different ledgers. The ordering service is the same across any network or channel.

A few things that must be kept in mind are:

- The network setup allows for the creation of channels.
- The same chaincode logic can be applied to multiple channels.

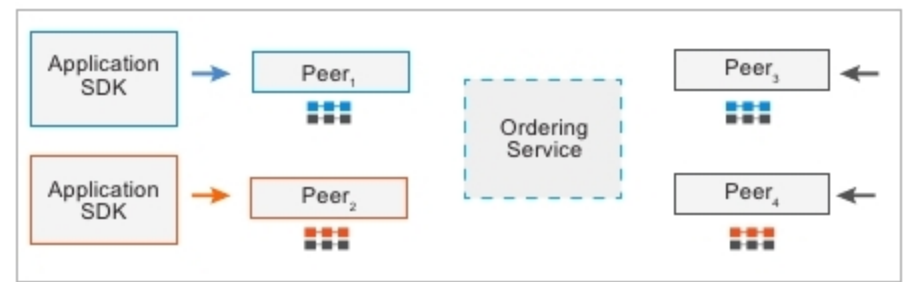


Figure 1: Key channels

- A given user can participate in multiple channels.

State database

The current state data represents the latest values for all assets in the ledger. Since the current state represents all the committed transactions on the channel, it is sometimes referred to as the world state. Chaincode invocations execute transactions against the current state data. To make these chaincode interactions extremely efficient, the latest key/value pairs for each asset are stored in a state database, which is simply an indexed view into the chain's committed transactions. It can therefore be regenerated from the chain at any time. The state database will automatically be recovered (or generated, if needed) upon peer startup, before new transactions are accepted. The default state database, LevelDB, can be replaced with CouchDB.

LevelDB is the default key/value state database for Hyperledger Fabric and simply stores key/value pairs. CouchDB is an alternative to LevelDB. Unlike LevelDB, CouchDB stores JSON objects. The latter is unique in that it supports keyed, composite, key range, and full data-rich queries.

Hyperledger Fabric's LevelDB and CouchDB are very similar in their structure and functions. Both support core chaincode operations, such as getting and setting key assets, and querying based on these keys. With both, keys can be queried by range and composite keys can be modelled to enable equivalence queries against multiple parameters. But, as a JSON document store, CouchDB additionally enables rich query against the chaincode data, when chaincode values (e.g., assets) are modelled as JSON data.

Smart contracts

As a reminder, smart contracts are computer programs that contain logic to execute transactions and modify the state of the assets stored within the ledger. Hyperledger Fabric's smart contracts are called chaincode and are written in Go. The chaincode serves as the business logic for a Hyperledger Fabric network, in that it directs how you manipulate assets within the network.

Membership service provider (MSP)

The membership service provider, or MSP, is a component that defines the rules in which identities are validated, authenticated and allowed access to a network. The MSP manages user IDs and authenticates clients who want to join the network. This includes providing credentials for